Assignment_2

This is an R Markdown Notebook. When you execute code within the notebook, the results appear beneath the code.

Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Ctrl+Shift+Enter*.

```
## Assignment 2 files can be located in my GitHub repository https://github.com/Dylan-Hamilton3412/64060_-Dhamil28

## Import required packages for data preparation, preprocessing,
## k-NN classification, and model evaluation.
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.5.2
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
## Loading required package: lattice
```

```
library(readr)
library(class)
library(FNN)
```

```
## Warning: package 'FNN' was built under R version 4.5.3
```

```
##
## Attaching package: 'FNN'
```

```
## The following objects are masked from 'package:class':
##
##     knn, knn.cv
```

```
library(gmodels)
```

```
## Warning: package 'gmodels' was built under R version 4.5.3
```

```r
## Set seed to ensure results are reproducible.
set.seed(123)

## Load the Universal Bank dataset and review the structure.
## Personal.Loan is the target variable to be predicted per the assignment in
structions
BankData <- read.csv("UniversalBank.csv")
summary(BankData)
```

```
##        ID            Age          Experience       Income          ZIP.Cod
e
##  Min.   :   1   Min.   :23.00   Min.   :-3.0   Min.   :  8.00   Min.   : 9
307
##  1st Qu.:1251   1st Qu.:35.00   1st Qu.:10.0   1st Qu.: 39.00   1st Qu.:91
911
##  Median :2500   Median :45.00   Median :20.0   Median : 64.00   Median :93
437
##  Mean   :2500   Mean   :45.34   Mean   :20.1   Mean   : 73.77   Mean   :93
153
##  3rd Qu.:3750   3rd Qu.:55.00   3rd Qu.:30.0   3rd Qu.: 98.00   3rd Qu.:94
608
##  Max.   :5000   Max.   :67.00   Max.   :43.0   Max.   :224.00   Max.   :96
651
##      Family          CCAvg          Education        Mortgage
##  Min.   :1.000   Min.   : 0.000   Min.   :1.000   Min.   :  0.0
##  1st Qu.:1.000   1st Qu.: 0.700   1st Qu.:1.000   1st Qu.:  0.0
##  Median :2.000   Median : 1.500   Median :2.000   Median :  0.0
##  Mean   :2.396   Mean   : 1.938   Mean   :1.881   Mean   : 56.5
##  3rd Qu.:3.000   3rd Qu.: 2.500   3rd Qu.:3.000   3rd Qu.:101.0
##  Max.   :4.000   Max.   :10.000   Max.   :3.000   Max.   :635.0
##  Personal.Loan   Securities.Account   CD.Account         Online
##  Min.   :0.000   Min.   :0.0000     Min.   :0.0000   Min.   :0.0000
##  1st Qu.:0.000   1st Qu.:0.0000     1st Qu.:0.0000   1st Qu.:0.0000
##  Median :0.000   Median :0.0000     Median :0.0000   Median :1.0000
##  Mean   :0.096   Mean   :0.1044     Mean   :0.0604   Mean   :0.5968
##  3rd Qu.:0.000   3rd Qu.:0.0000     3rd Qu.:0.0000   3rd Qu.:1.0000
##  Max.   :1.000   Max.   :1.0000     Max.   :1.0000   Max.   :1.0000
##    CreditCard
##  Min.   :0.000
##  1st Qu.:0.000
##  Median :0.000
##  Mean   :0.294
##  3rd Qu.:1.000
##  Max.   :1.000
```

```r
str(BankData)
```

```
## 'data.frame':    5000 obs. of  14 variables:
##  $ ID                : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ Age               : int  25 45 39 35 35 37 53 50 35 34 ...
```

```
##  $ Experience        : int  1 19 15 9 8 13 27 24 10 9 ...
##  $ Income            : int  49 34 11 100 45 29 72 22 81 180 ...
##  $ ZIP.Code          : int  91107 90089 94720 94112 91330 92121 91711 9394
3 90089 93023 ...
##  $ Family            : int  4 3 1 1 4 4 2 1 3 1 ...
##  $ CCAvg             : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
##  $ Education         : int  1 1 1 2 2 2 2 3 2 3 ...
##  $ Mortgage          : int  0 0 0 0 0 155 0 0 104 0 ...
##  $ Personal.Loan     : int  0 0 0 0 0 0 0 0 0 1 ...
##  $ Securities.Account: int  1 1 0 0 0 0 0 0 0 0 ...
##  $ CD.Account        : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ Online            : int  0 0 0 0 0 1 1 0 1 0 ...
##  $ CreditCard        : int  0 0 0 0 1 0 0 1 0 0 ...

## Remove ID and ZIP Code because they do not provide meaningful predictive v
alue and could distort distance calculations.
BankData_1 <- select(BankData, -ZIP.Code, -ID)

## Convert Education to a factor because it is a categorical variable.
BankData_1$Education <- as.factor(BankData_1$Education)

## Create a 60% training set and 40% validation set.
## Stratified sampling preserves the class distribution.
Train_Index_1 <- createDataPartition(BankData_1$Personal.Loan, p = .6, list=F
ALSE)

## k-NN requires numeric inputs. Therefore, Education is converted into dummy
 variables using the training data only.
Train_BankData_1 <- BankData_1[Train_Index_1,]
edu_dummy <- dummyVars(~ Education, data = Train_BankData_1)
edu_train <- predict(edu_dummy, newdata = Train_BankData_1)
Train_BankData_1 <- cbind(Train_BankData_1, edu_train)
## Remove the original Education variable after creating dummies.
Train_BankData_1$Education <- NULL

## Using the Same process as above and apply to validation set
Val_BankData_1 <- BankData_1[-Train_Index_1,]
edu_test <- predict(edu_dummy, newdata = Val_BankData_1)
Val_BankData_1 <- cbind(Val_BankData_1, edu_test)
Val_BankData_1$Education <- NULL

## Separate predictor variables from the target variable. Personal.Loan shoul
d not be normalized since it is the target variable
Train_Norm_1 <- select(Train_BankData_1, -Personal.Loan)
Val_Norm_1<- select(Val_BankData_1, -Personal.Loan)

## Creating set with only the target variable
Train_PL_1 <- as.factor(Train_BankData_1$Personal.Loan)
Val_PL_1 <- as.factor(Val_BankData_1$Personal.Loan)
```

```r
## Normalize all predictor variables using z-score standardization.
## Normalization is important for k-NN because distance calculations are sens
itive to variable scale.
norm.val <- preProcess(Train_Norm_1, method = c("center","scale"))

## Apply training-set normalization parameters to both datasets.
Train_Norm_1 <- predict(norm.val, Train_Norm_1)
Val_Norm_1 <- predict(norm.val, Val_Norm_1)
## By using the summary below we can identify the normalization acted appropr
iately by the means of the variables being 0
summary(Train_Norm_1)
```

```
##       Age               Experience          Income             Family
##  Min.   :-1.96926   Min.   :-2.03259   Min.   :-1.4286   Min.   :-1.2142
##  1st Qu.:-0.82784   1st Qu.:-0.89392   1st Qu.:-0.7514   1st Qu.:-1.2142
##  Median : 0.05016   Median :-0.01801   Median :-0.2053   Median :-0.3463
##  Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.: 0.84037   3rd Qu.: 0.85789   3rd Qu.: 0.4720   3rd Qu.: 1.3895
##  Max.   : 1.89398   Max.   : 1.99656   Max.   : 3.2901   Max.   : 1.3895
##      CCAvg             Mortgage       Securities.Account   CD.Account
##  Min.   :-1.1157   Min.   :-0.5477   Min.   :-0.3388    Min.   :-0.2504
##  1st Qu.:-0.7098   1st Qu.:-0.5477   1st Qu.:-0.3388    1st Qu.:-0.2504
##  Median :-0.2459   Median :-0.5477   Median :-0.3388    Median :-0.2504
##  Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000    Mean   : 0.0000
##  3rd Qu.: 0.3921   3rd Qu.: 0.4231   3rd Qu.:-0.3388    3rd Qu.:-0.2504
##  Max.   : 4.6835   Max.   : 5.7429   Max.   : 2.9506    Max.   : 3.9930
##      Online          CreditCard       Education.1       Education.2
##  Min.   :-1.2237   Min.   :-0.6457   Min.   :-0.8578   Min.   :-0.6312
##  1st Qu.:-1.2237   1st Qu.:-0.6457   1st Qu.:-0.8578   1st Qu.:-0.6312
##  Median : 0.8169   Median :-0.6457   Median :-0.8578   Median :-0.6312
##  Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.: 0.8169   3rd Qu.: 1.5481   3rd Qu.: 1.1653   3rd Qu.: 1.5836
##  Max.   : 0.8169   Max.   : 1.5481   Max.   : 1.1653   Max.   : 1.5836
##   Education.3
##  Min.   :-0.6405
##  1st Qu.:-0.6405
##  Median :-0.6405
##  Mean   : 0.0000
##  3rd Qu.: 1.5606
##  Max.   : 1.5606
```

```r
## Test my datasets to ensure that output predications match expectations
Predicted_N_Labels <- knn(Train_Norm_1, Val_Norm_1,Train_PL_1, prob = FALSE,
k=1)
head(Predicted_N_Labels)
```

```
## [1] 0 0 0 0 0 1
## Levels: 0 1
```

```r
## Question 1
## Classify the specified customer using k = 1.
Customer_1 <- data.frame(Age = 40, Experience = 10, Income = 84, Family = 2,
CCAvg = 2, Mortgage = 0, Securities.Account = 0, CD.Account = 0, Online = 1,
CreditCard = 1, Education.1 = 0, Education.2 = 1, Education.3 = 0)

## Apply the same normalization used on the training data.
Customer_1 <- predict(norm.val, Customer_1)
Customer_1_Class <- knn(Train_Norm_1, Customer_1, Train_PL_1, prob = FALSE, k
=1)
Customer_1_Class
```

```
## [1] 0
## attr(,"nn.index")
##      [,1]
## [1,] 2647
## attr(,"nn.dist")
##           [,1]
## [1,] 0.4993493
## Levels: 0
```

```r
## Result:
## Customer classified as: 0
## Interpretation: The nearest customer in the training set did not
## accept the loan offer, therefore this customer is predicted not
## to accept the loan.

## Question 2
## Evaluate k values from 1 to 20 and identify the value that
## provides the best balance between overfitting and underfitting.
Knn_Accuracy <- data.frame(k = seq(1, 20, 1), accuracy = rep(0, 20))
for(i in 1:20) {
  knn.pred <- knn(Train_Norm_1, Val_Norm_1, Train_PL_1, prob = FALSE, k=i)
  Knn_Accuracy[i, 2] <- confusionMatrix(knn.pred, Val_PL_1)$overall[1]
}

Knn_Accuracy
```

```
##     k accuracy
## 1   1   0.9635
## 2   2   0.9520
## 3   3   0.9585
## 4   4   0.9485
## 5   5   0.9545
## 6   6   0.9515
## 7   7   0.9545
## 8   8   0.9460
## 9   9   0.9495
## 10 10   0.9450
## 11 11   0.9465
```

```
## 12 12   0.9450
## 13 13   0.9465
## 14 14   0.9435
## 15 15   0.9435
## 16 16   0.9415
## 17 17   0.9435
## 18 18   0.9405
## 19 19   0.9420
## 20 20   0.9390

## Insight:
## Even though the highest accuracy was k=1 Small k values are sensitive to n
oise and may overfit.
## Large k values smooth the decision boundary and may underfit.
## Based on validation accuracy, k = 1 however based off the possibility of o
verfitting the most realistic is k = 3 with
## an accuracy of .9585 to compromise between these extremes.

## Question 3
## Generate a confusion matrix using the selected value of k.
Conf_Mat_Pred <- knn(Train_Norm_1, Val_Norm_1,Train_PL_1, prob = FALSE, k=3)
CrossTable(Val_PL_1, Conf_Mat_Pred, prop.chisq = FALSE)

##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |           N / Row Total |
## |           N / Col Total |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  2000
##
##
##                | Conf_Mat_Pred
##     Val_PL_1 |         0 |         1 | Row Total |
## -------------|-----------|-----------|-----------|
##            0 |      1787 |        11 |      1798 |
##              |     0.994 |     0.006 |     0.899 |
##              |     0.961 |     0.078 |           |
##              |     0.893 |     0.005 |           |
## -------------|-----------|-----------|-----------|
##            1 |        72 |       130 |       202 |
##              |     0.356 |     0.644 |     0.101 |
##              |     0.039 |     0.922 |           |
##              |     0.036 |     0.065 |           |
## -------------|-----------|-----------|-----------|
```

```
## Column Total |      1859 |       141 |      2000 |
##              |     0.929 |     0.070 |           |
## -------------|-----------|-----------|-----------|
##
##
```

```
## Insight:
## The confusion matrix shows the model correctly identifies
## most customers who reject the loan while maintaining reasonable
## performance in identifying customers who accept the loan.
```

```
## Question 4
## Reclassify the same customer using the selected value of k = 3.
Customer_K3_Class <- knn(Train_Norm_1,Customer_1,Train_PL_1, prob = FALSE, k
= 3)
Customer_K3_Class
```

```
## [1] 0
## attr(,"nn.index")
##      [,1] [,2] [,3]
## [1,] 2647 2040  959
## attr(,"nn.dist")
##           [,1]      [,2]      [,3]
## [1,] 0.4993493 0.6383913 0.7102609
## Levels: 0
```

```
## Result:
## Customer classified as: 0
## Even after considering the three nearest neighbors, the majority
## vote predicts that the customer will not accept the loan offer.
```

```
## Question 5:
## Repartition the data into Training (50%), Validation (30%),
## and Test (20%) datasets.
```

```
## The below partitioning, normalization and transformations are all complete
d identically to the first
## section of this code this is why no additional comments are made to descri
be the process
```

```
BankData_2 <- select(BankData, -ZIP.Code, -ID)
BankData_2$Education <- as.factor(BankData_2$Education)
```

```
## First create the test set (20%).
## Then split the remaining 80% into training and validation sets.
Test_Index_2 <- createDataPartition(BankData_2$Personal.Loan, p = .2, list=FA
LSE)
Test_Data_2 <- BankData_2[Test_Index_2,]
TrainVal_Data_2 <- BankData_2[-Test_Index_2,]
```

```r
Train_Index_2 <- createDataPartition(TrainVal_Data_2$Personal.Loan, p = .625,
 list=FALSE)
Train_Data_2 <- TrainVal_Data_2[Train_Index_2,]
Val_Data_2 <- TrainVal_Data_2[-Train_Index_2,]

edu_dummy_2 <- dummyVars(~ Education, data = Train_Data_2)

edu_Test_2 <- predict(edu_dummy_2, newdata = Test_Data_2)
edu_Train_2 <- predict(edu_dummy_2, newdata = Train_Data_2)
edu_Val_2 <- predict(edu_dummy_2, newdata = Val_Data_2)

Test_Data_2 <- cbind(Test_Data_2, edu_Test_2)
Train_Data_2 <- cbind(Train_Data_2, edu_Train_2)
Val_Data_2 <- cbind(Val_Data_2, edu_Val_2)

Test_Data_2$Education = NULL
Train_Data_2$Education = NULL
Val_Data_2$Education = NULL

Test_Norm_2 <- select(Test_Data_2, -Personal.Loan)
Train_Norm_2 <- select(Train_Data_2, -Personal.Loan)
Val_Norm_2 <- select(Val_Data_2, -Personal.Loan)

Test_PL_2 <- as.factor(Test_Data_2$Personal.Loan)
Train_PL_2 <- as.factor(Train_Data_2$Personal.Loan)
Val_PL_2 <- as.factor(Val_Data_2$Personal.Loan)

norm.val_2 <- preProcess(Train_Norm_2, method = c("center","scale"))
Test_Norm_2 <- predict(norm.val_2, Test_Norm_2)
Train_Norm_2 <- predict(norm.val_2, Train_Norm_2)
Val_Norm_2 <- predict(norm.val_2, Val_Norm_2)

## By using the summary below we can identify the normalization acted appropr
iately by the means of the variables being 0
summary(Train_Norm_2)
```

```
##       Age               Experience           Income             Family

##  Min.   :-1.949260   Min.   :-2.02146   Min.   :-1.4391   Min.   :-1.1885

##  1st Qu.:-0.890570   1st Qu.:-0.87248   1st Qu.:-0.7509   1st Qu.:-1.1885

##  Median :-0.008328   Median : 0.01135   Median :-0.2181   Median :-0.3179

##  Mean   : 0.000000   Mean   : 0.00000   Mean   : 0.0000   Mean   : 0.0000

##  3rd Qu.: 0.873913   3rd Qu.: 0.80679   3rd Qu.: 0.4924   3rd Qu.: 0.5526
```

```
##  Max.   : 1.932603   Max.   : 1.95577   Max.   : 2.9123   Max.   : 1.4231

##      CCAvg            Mortgage       Securities.Account   CD.Account
##  Min.   :-1.1433   Min.   :-0.5486   Min.   :-0.3362    Min.   :-0.2417
##  1st Qu.:-0.7246   1st Qu.:-0.5486   1st Qu.:-0.3362    1st Qu.:-0.2417
##  Median :-0.1862   Median :-0.5486   Median :-0.3362    Median :-0.2417
##  Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000    Mean   : 0.0000
##  3rd Qu.: 0.3522   3rd Qu.: 0.4460   3rd Qu.:-0.3362    3rd Qu.:-0.2417
##  Max.   : 4.4199   Max.   : 5.5271   Max.   : 2.9730    Max.   : 4.1363
##      Online          CreditCard       Education.1       Education.2
##  Min.   :-1.2073   Min.   :-0.6551   Min.   :-0.8314   Min.   :-0.6365
##  1st Qu.:-1.2073   1st Qu.:-0.6551   1st Qu.:-0.8314   1st Qu.:-0.6365
##  Median : 0.8279   Median :-0.6551   Median :-0.8314   Median :-0.6365
##  Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.: 0.8279   3rd Qu.: 1.5258   3rd Qu.: 1.2023   3rd Qu.: 1.5705
##  Max.   : 0.8279   Max.   : 1.5258   Max.   : 1.2023   Max.   : 1.5705
##   Education.3
##  Min.   :-0.6589
##  1st Qu.:-0.6589
##  Median :-0.6589
##  Mean   : 0.0000
##  3rd Qu.: 1.5171
##  Max.   : 1.5171
```

```
Test_2_Class <- knn(Train_Norm_2, Test_Norm_2, Train_PL_2, prob = FALSE, k=3)
Train_2_Class <- knn(Train_Norm_2, Train_Norm_2, Train_PL_2, prob = FALSE, k=
3)
Val_2_Class <- knn(Train_Norm_2, Val_Norm_2, Train_PL_2, prob = FALSE, k=3)

## Calculate performance metrics for comparison across datasets.
## Recall measures the ability to identify loan acceptors.
## Precision measures the proportion of predicted acceptors that actually acc
epted the loan.
## TNR measures the ability to identify non-acceptors.
## FPR measures the rate of false positives.

CrossTable(Test_PL_2, Test_2_Class, prop.chisq = FALSE)

##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |           N / Row Total |
## |           N / Col Total |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  1000
```

```
##
##
##               | Test_2_Class
##    Test_PL_2 |         0 |         1 | Row Total |
## -------------|-----------|-----------|-----------|
##            0 |       902 |         6 |       908 |
##              |     0.993 |     0.007 |     0.908 |
##              |     0.966 |     0.091 |           |
##              |     0.902 |     0.006 |           |
## -------------|-----------|-----------|-----------|
##            1 |        32 |        60 |        92 |
##              |     0.348 |     0.652 |     0.092 |
##              |     0.034 |     0.909 |           |
##              |     0.032 |     0.060 |           |
## -------------|-----------|-----------|-----------|
## Column Total |       934 |        66 |      1000 |
##              |     0.934 |     0.066 |           |
## -------------|-----------|-----------|-----------|
##
##
```

```
Test_Recall <- 60/92
Test_Prec <- 60/66
Test_TNR <- 902/934
Test_FPR <- 1-Test_TNR
```

```
CrossTable(Train_PL_2, Train_2_Class, prop.chisq = FALSE)
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |           N / Row Total |
## |           N / Col Total |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  2500
##
##
##               | Train_2_Class
##   Train_PL_2 |         0 |         1 | Row Total |
## -------------|-----------|-----------|-----------|
##            0 |      2260 |         6 |      2266 |
##              |     0.997 |     0.003 |     0.906 |
##              |     0.977 |     0.032 |           |
##              |     0.904 |     0.002 |           |
## -------------|-----------|-----------|-----------|
```

```
##             1 |        53 |       181 |       234 |
##               |     0.226 |     0.774 |     0.094 |
##               |     0.023 |     0.968 |           |
##               |     0.021 |     0.072 |           |
## -------------|-----------|-----------|-----------|
## Column Total |      2313 |       187 |      2500 |
##               |     0.925 |     0.075 |           |
## -------------|-----------|-----------|-----------|
##
##

Train_Recall <- 181/234
Train_Prec <- 181/187
Train_TNR <- 2260/2313
Train_FPR <- 1-Train_TNR

CrossTable(Val_PL_2, Val_2_Class, prop.chisq = FALSE)

##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |           N / Row Total |
## |           N / Col Total |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  1500
##
##
##              | Val_2_Class
##     Val_PL_2 |         0 |         1 | Row Total |
## -------------|-----------|-----------|-----------|
##            0 |      1333 |        13 |      1346 |
##              |     0.990 |     0.010 |     0.897 |
##              |     0.962 |     0.114 |           |
##              |     0.889 |     0.009 |           |
## -------------|-----------|-----------|-----------|
##            1 |        53 |       101 |       154 |
##              |     0.344 |     0.656 |     0.103 |
##              |     0.038 |     0.886 |           |
##              |     0.035 |     0.067 |           |
## -------------|-----------|-----------|-----------|
## Column Total |      1386 |       114 |      1500 |
##              |     0.924 |     0.076 |           |
## -------------|-----------|-----------|-----------|
##
##
```

```r
Val_Recall <- 101/154
Val_Prec <-101/114
Val_TNR <- 1333/1386
Val_FPR <- 1-Val_TNR

## For easier comparison of the confusion matrix values I created a table usi
ng the recall, precision
## true negative rate, and false positive rate
CFM_Metrics <- data.frame(
  Dataset = c("Train", "Validation", "Test"),
  Recall = c(Train_Recall, Val_Recall, Test_Recall),
  Precision = c(Train_Prec, Val_Prec, Test_Prec),
  TNR = c(Train_TNR, Val_TNR, Test_TNR),
  FPR = c(Train_FPR, Val_FPR, Test_FPR)
)

CFM_Metrics
```

```
##       Dataset    Recall Precision       TNR        FPR
## 1       Train 0.7735043 0.9679144 0.9770860 0.02291396
## 2 Validation 0.6558442 0.8859649 0.9617605 0.03823954
## 3        Test 0.6521739 0.9090909 0.9657388 0.03426124
```

```r
## Insight:
## Training performance is slightly better than validation and test
## performance because the model was built using the training data.
##
## Validation and test metrics are very similar, indicating that the
## model generalizes reasonably well to unseen customers.
##
## The relatively high precision and TNR values suggest the model is
## effective at identifying customers who are unlikely to accept a
## loan offer. Recall is lower, indicating some potential loan
## acceptors are still being missed.
```

Add a new chunk by clicking the *Insert Chunk* button on the toolbar or by pressing *Ctrl+Alt+I*.

When you save the notebook, an HTML file containing the code and output will be saved alongside it (click the *Preview* button or press *Ctrl+Shift+K* to preview the HTML file).

The preview shows you a rendered HTML copy of the contents of the editor. Consequently, unlike *Knit*, *Preview* does not run any R code chunks. Instead, the output of the chunk when it was last run in the editor is displayed.